

THE ARCHITECTURE AND IMPLEMENTATION

The architecture and the implementation plan of an IoT telemetry for green house monitoring is being described in this document. The system composed of asynchronous sensor simulators, REST services, a telemetry ingestion server and WebSocket live feeds

A. System architecture

The system is composed of three major parts: the sensor simulator, the telemetry server, and the client applications.

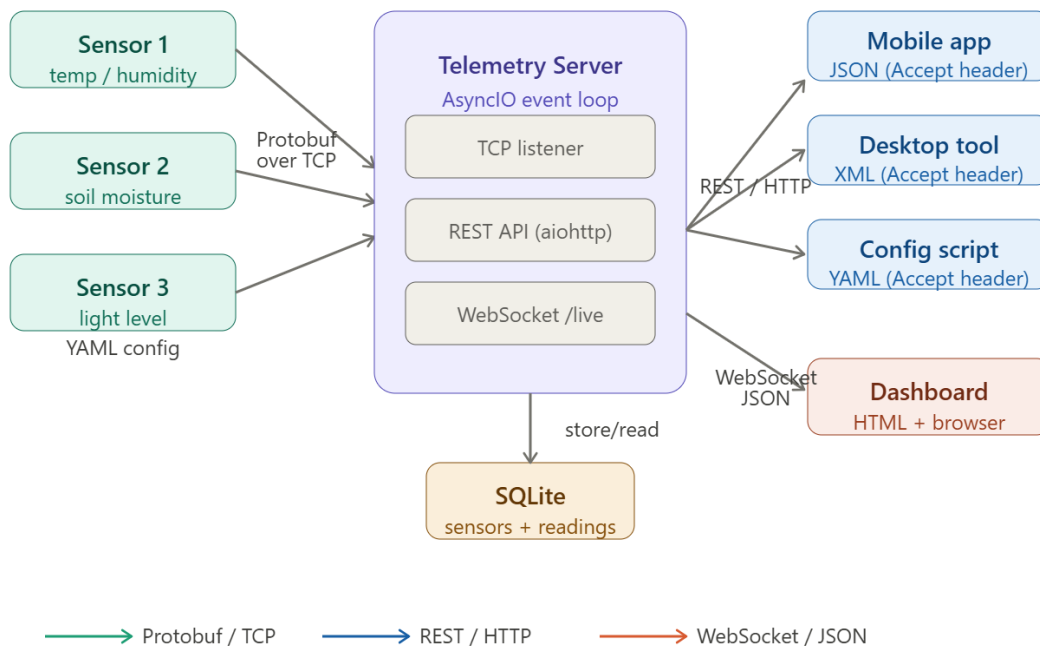
The sensor simulator generates fake greenhouse readings such as temperature, humidity, and soil moisture. These readings are serialized using Protocol Buffers (Protobuf) and sent to the telemetry server over TCP sockets.

The telemetry server receives incoming sensor data asynchronously using AsyncIO. The server stores the readings and exposes them through two interfaces:

1. A REST API for historical queries.
2. A WebSocket endpoint for live telemetry updates.

REST clients communicate with the server using HTTP and may request responses in JSON, XML, or YAML formats through content negotiation using the Accept header. Web browsers connect to the

WebSocket endpoint and receive real-time telemetry updates encoded as JSON.



B Sequence diagram explanation

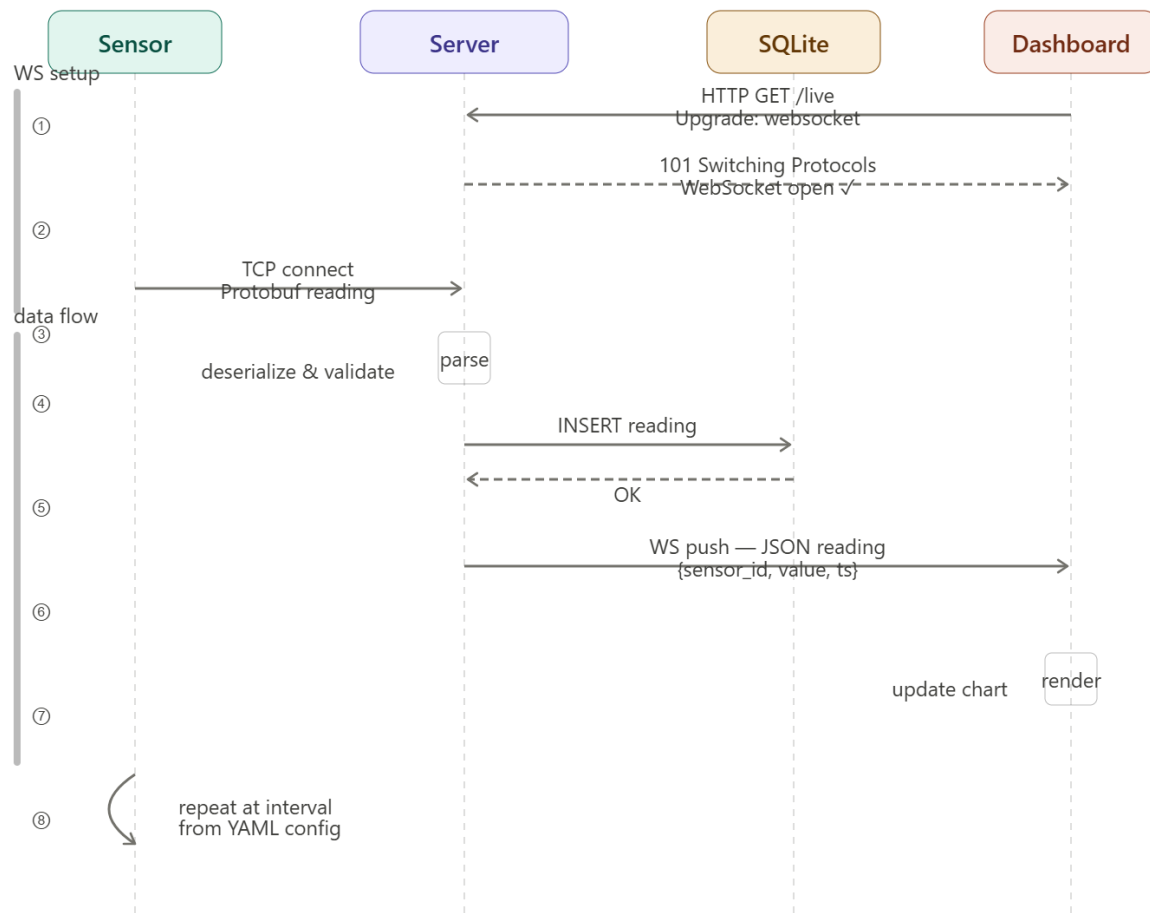
The sequence begins when a simulated sensor establishes a TCP connection with the telemetry server.

The sensor generates a telemetry reading and serializes it using Protobuf. The message is sent with a 4-byte length prefix to allow message framing over the TCP stream.

The telemetry server receives the message, decodes the Protobuf payload, and stores the reading in the storage layer.

The server then forwards the reading to the WebSocket broadcaster. Connected dashboard clients receive the update as a JSON message and immediately display the new reading.

Before WebSocket communication begins, the client performs an HTTP Upgrade handshake. The server responds with “101 Switching Protocols,” after which the persistent WebSocket connection is established.



The WebSocket upgrade happens first (steps ① and ②), then sensors continuously send Protobuf readings (③), the server parses and stores them (④–⑤), then pushes to all connected dashboards via WebSocket (⑥–⑦).

C.Data model

The system uses two main entities: Sensor and Reading.

A Sensor represents a telemetry-producing device such as a temperature or humidity sensor. Each sensor has a unique identifier and a sensor type.

A Reading represents one telemetry measurement generated by a sensor. Each reading stores:

- sensor identifier
- reading type
- numeric value
- timestamp

The Reading structure is defined in the telemetry.proto schema and serialized using Protocol Buffers for efficient network transmission.

telemetry.proto Protobuf Copy

```
syntax = "proto3";
package telemetry;

// A single sensor reading sent from sensor to server over TCP
message Reading {
  string sensor_id = 1; // e.g. "sensor_01"
  float value = 2; // numeric reading
  string unit = 3; // "celsius", "percent", "lux"
  int64 timestamp = 4; // Unix epoch seconds
}

// Sensor registration information
message SensorInfo {
  string sensor_id = 1;
  string name = 2; // human-readable label
  string location = 3; // e.g. "Farm 1, Greenhouse A"
  string type = 4; // "temperature" | "humidity" | "soil_moisture" | "light"
  float interval = 5; // seconds between readings
}
```

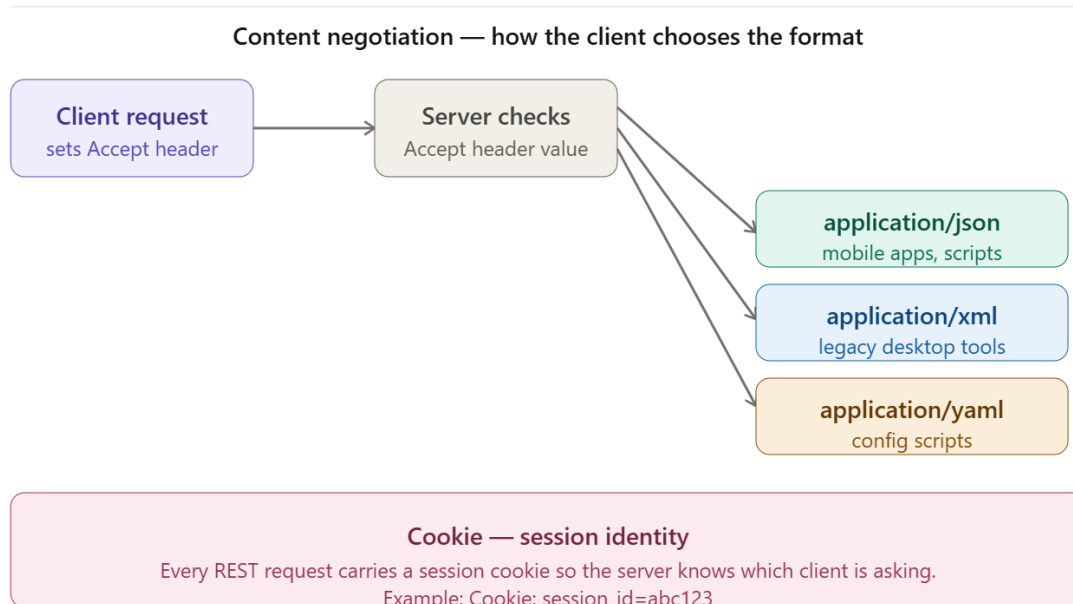
D.REST API

The REST API exposes historical telemetry data and sensor management functionality.

The client make a request, the API goes to the database or server and bring back the response. Each request is independent the API does remember the previous request.

The 4 Endpoints

GET	/sensors	Returns a list of all registered sensors
GET	/sensors/{id}/readings?from=...&to=...	Historical readings with time filter
POST	/sensors	Register a new sensor
DELETE	/sensors/{id}	Remove a sensor from the system



The API has four endpoints, each doing a specific job. GET /sensors retrieves a list of every sensor registered in the system. GET /sensors/{id}/readings retrieves the past readings for one particular sensor; the client can narrow the results to a time window by adding from and to query parameters (e.g. ?from=1746700000&to=1746786400). POST /sensors lets a client register a brand-new sensor with the system. DELETE /sensors/{id} removes a sensor permanently.

A key feature of this API is content negotiation. Rather than always returning the same format, the server looks at the Accept header in each incoming request and responds in the format the client asked for — JSON for mobile apps, XML for legacy desktop tools, or YAML for configuration scripts. This means the same endpoint serves every kind of client without any code changes on the server.

Every request also carries a session cookie (session_id). This is how the server tracks which client is making the request, without asking the client to log in every single time.

E.YAML configuration

The sensor simulator loads its configuration from a YAML file.

The configuration defines:

- the telemetry server host and port
- the list of sensors
- reporting intervals for each sensor

YAML was chosen because it is human-readable and easy to edit manually.

Config/sensor.yaml file

sensors.yaml YAML

 Copy

```
# NET322 Telemetry Pipeline – Sensor Configuration
# Each entry under 'sensors' defines one simulated sensor.
# The simulator reads this file on startup and runs one asyncio task per sensor.
```

```
sensors:
  - sensor_id: sensor_01
    name: Greenhouse A - Temperature
    location: Farm 1
    type: temperature
    unit: celsius
    interval: 5.0          # seconds between readings
    server_host: 127.0.0.1
    server_port: 9000

  - sensor_id: sensor_02
    name: Greenhouse A - Humidity
    location: Farm 1
    type: humidity
    unit: percent
    interval: 10.0
    server_host: 127.0.0.1
    server_port: 9000

  - sensor_id: sensor_03
    name: Greenhouse B - Soil Moisture
    location: Farm 2
    type: soil_moisture
    unit: percent
    interval: 15.0
    server_host: 127.0.0.1
    server_port: 9000
```

F. Concurrency model

The system uses AsyncIO for all networking and concurrency operations.

Each simulated sensor runs as an independent coroutine task. The telemetry server uses `asyncio.start_server()` to manage many simultaneous TCP connections without threads.

The REST API uses `aiohttp` and operates asynchronously on the same event loop.

The WebSocket server maintains one coroutine per connected browser client.

AsyncIO was selected because the application is highly I/O-bound rather than CPU-bound.

Failure handling

If a sensor disconnects unexpectedly, the telemetry server logs the event and continues serving other clients. The sensor simulator automatically retries the connection after a short delay.

Malformed Protobuf messages are handled using exception handling to prevent the server from crashing.

If the storage layer fails, the server logs the error and rejects additional writes while remaining operational where possible.

Slow WebSocket consumers are disconnected if they cannot keep up with live telemetry updates.

G. Failure handling

When the sensor disconnects unexpectedly, the telemetry server logs the event and continues serving other clients. The sensor simulator automatically retries the connection after a short delay.

Malformed Protobuf messages are handled using exception handling. The server wraps `ParseString()` in a `try` or `except`, logs the error and closes that connection. to prevent the server from crashing.

If the storage layer fails, the server logs the error and rejects additional writes while remaining operational where possible.

Slow WebSocket consumers are disconnected if they cannot keep up with live telemetry updates.

References

Google, “Protocol Buffers Documentation.”

[Protocol Buffers Official Documentation](#)

[Python AsyncIO Documentation](#)[aiohttp Official Documentation](#)[websockets Documentation](#)

YAML, “YAML Ain’t Markup Language (YAML).”

[YAML Official Website](#)

Mozilla Foundation, “HTTP Content Negotiation.”

MDN Content Negotiation Guide

Mozilla Foundation, “WebSocket API.”

[MDN WebSocket Documentation](#)

IETF, “The WebSocket Protocol (RFC 6455).”

[RFC 6455 WebSocket Protocol](#)

IETF, “HTTP Semantics (RFC 9110).”

[RFC 9110 HTTP Semantics](#)

Designing Data-Intensive Applications, Martin Kleppmann, *Designing Data-Intensive Applications*, O’Reilly Media, 2017.

Fluent Python, Luciano Ramalho, *Fluent Python*, 2nd Edition, O’Reilly Media, 2022.

Python Concurrency with asyncio, Matthew Fowler, *Python Concurrency with asyncio*, Publications, 2022.